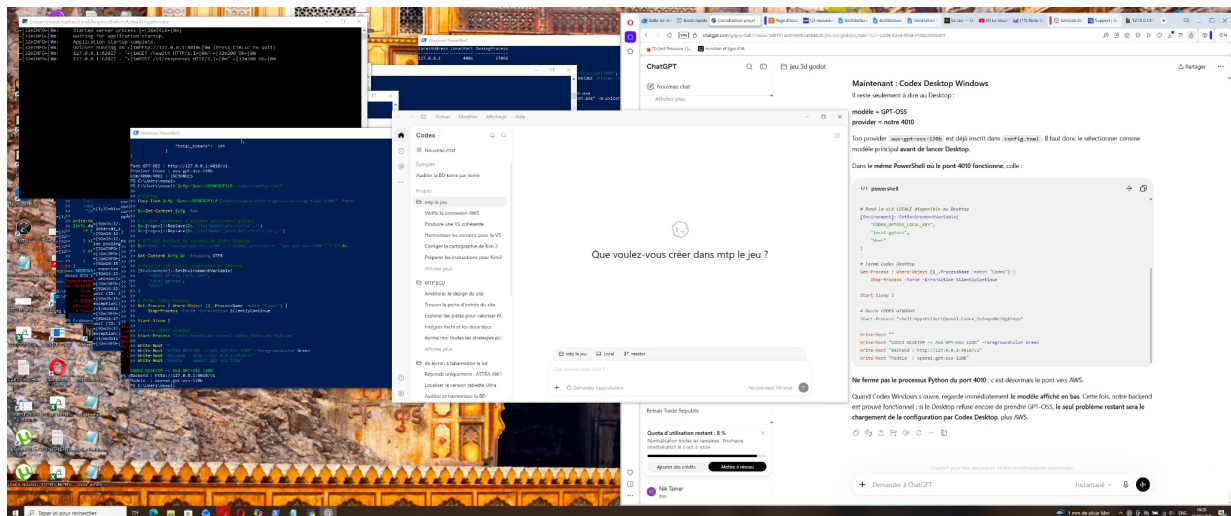


FRANKENSTEIN

Codex Desktop x AWS Bedrock Mantle x GPT-OSS 120B

Dossier premium - architecture, raisonnement et preuve de fonctionnement

26 septembre 2026



Capture de validation finale : le serveur local 4010, Codex Desktop et la reponse d'inference AWS reunis dans le meme environnement.

Un montage hybride assume : l'atelier reste Codex, le pont est local, le cloud est AWS et le cerveau distant est GPT-OSS 120B.

1. L'objectif : changer le cerveau de l'agent

Le projet avait déjà obtenu une connexion AWS MCP dans Codex. Mais MCP ne remplace pas le modèle qui raisonne : il fournit des outils. L'objectif de Frankenstein était différent : conserver Codex Desktop Windows comme environnement de travail tout en déportant l'inférence vers un gros modèle AWS distinct.

Architecture cible

```
Codex Desktop Windows
|
| Responses API
v
127.0.0.1:4010/v1
|
| proxy FastAPI/httpx
v
AWS Bedrock Mantle - us-east-1
|
v
openai.gpt-oss-120b
```

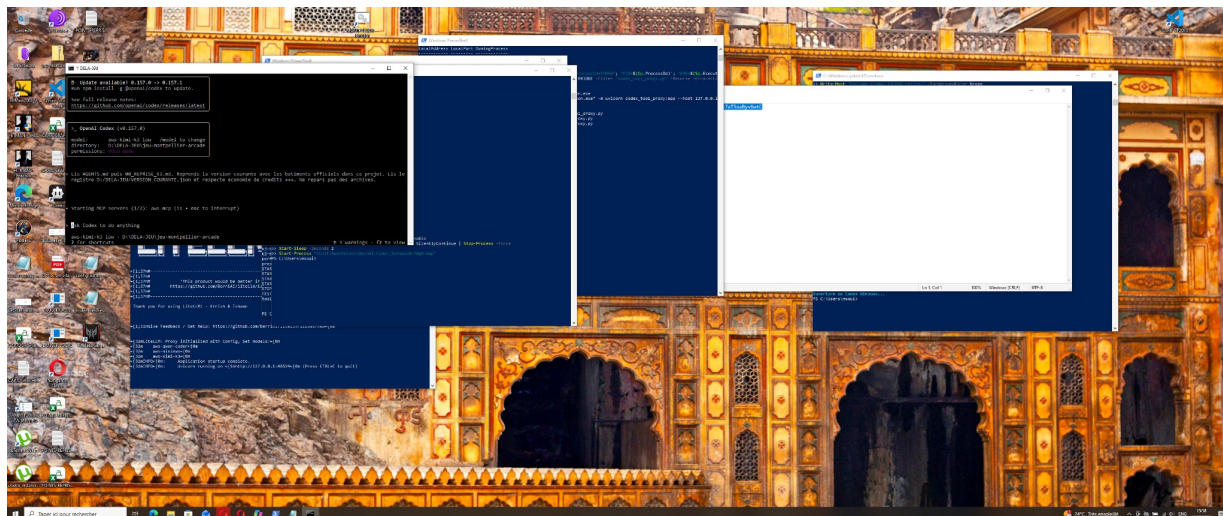
Cette distinction explique tout le raisonnement qui suit : ne pas confondre interface, outils MCP, provider Codex, proxy local et modèle d'inférence.

Pourquoi GPT-OSS 120B

Le contrôle du compte AWS avait établi que GPT-OSS 120B était disponible sur Mantle. Son intérêt ici est sa vocation généraliste : raisonnement, code et usage agentique. Il devient l'ingénieur généraliste, tandis que le codeur spécialisé existant peut rester concentré sur la production.

2. Le premier Frankenstein : le bridge historique

Le projet disposait déjà d'un pont AWS multi-modèles. Codex envoyait ses requêtes au proxy 4001, qui normalisait les noms d'outils puis relayait vers LiteLLM sur 4000. LiteLLM devait ensuite router vers AWS Bedrock.



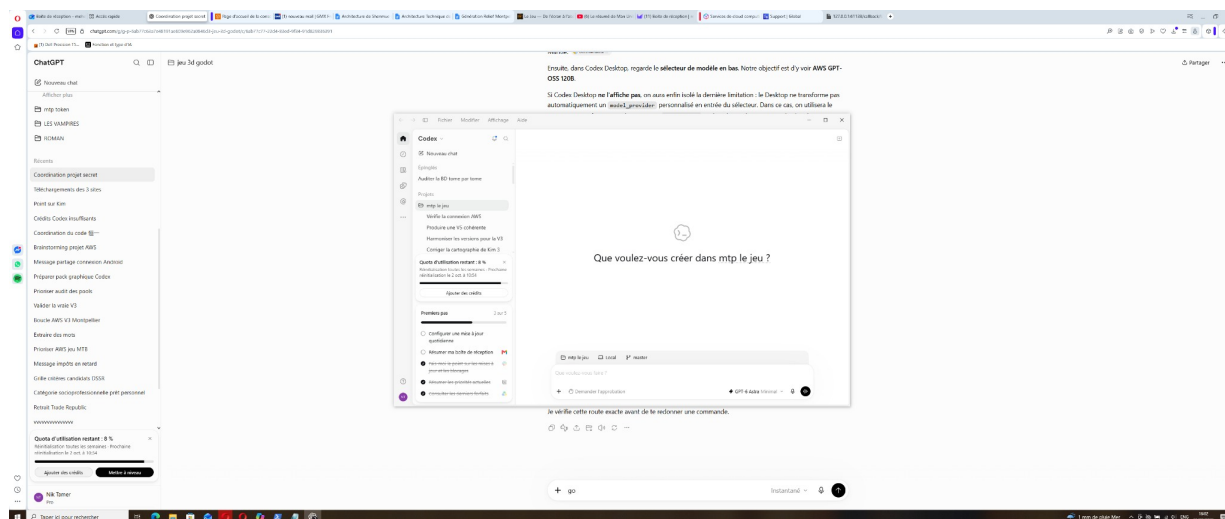
Le bridge historique en fonctionnement : consoles locales, Codex et processus AWS coexistent déjà sur la machine.

Codex -> :4001 codex_tool_proxy.py -> :4000 LiteLLM -> AWS Bedrock

Ce montage n'est pas jete. Il reste la voie des agents historiques et contient une logique utile de normalisation/restauration des noms d'outils.

3. Premier echec : le Desktop s'ouvre, mais Astra reste visible

Une tentative a consiste a declarer directement le provider Mantle dans config.toml et a relancer Codex Desktop. Le lancement de l'application reussissait, mais l'interface restait sur Astra. Cette etape a montre qu'ouvrir Codex et definir un provider ne suffisent pas a prouver le routage reel du modele.

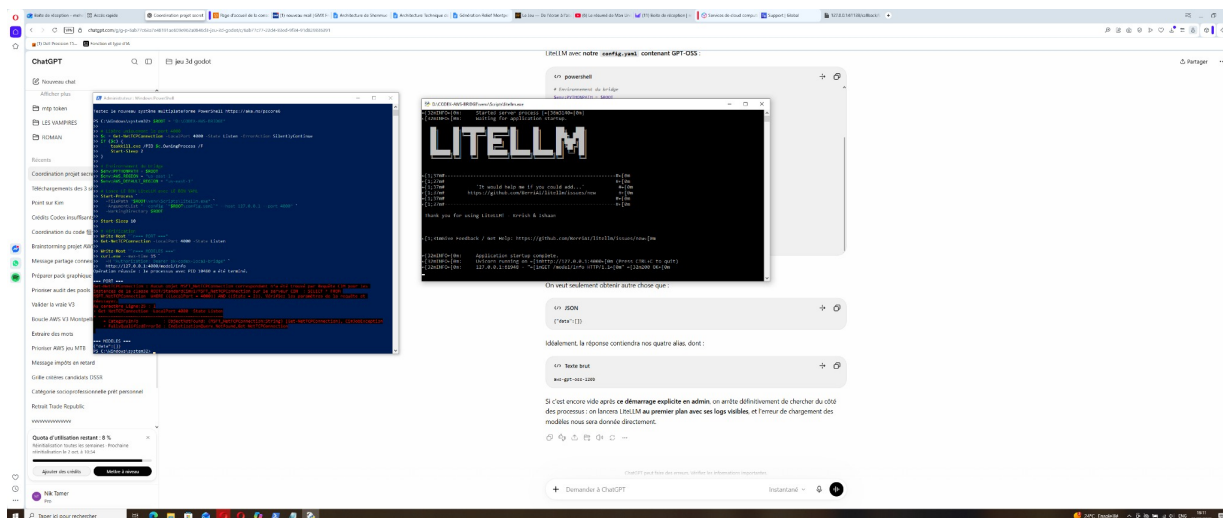


Etape intermediaire : Codex Desktop est relance, mais le selecteur affiche encore Astra. Le probleme est desormais separe de l'ouverture de l'application.

Conclusion de cette etape : il faut prouver l'inference independamment de l'etiquette affichee par l'interface.

4. Le faux coupable : GPT-OSS n'etait pas le probleme

Le fichier config.yaml contenait correctement les routes aws-qwen-coder, aws-minimax, aws-kimi-k3 et aws-gpt-oss-120b. Pourtant LiteLLM repondait avec une liste de modeles vide. Le serveur demarrait, mais n'enregistrait aucun modele exploitable.



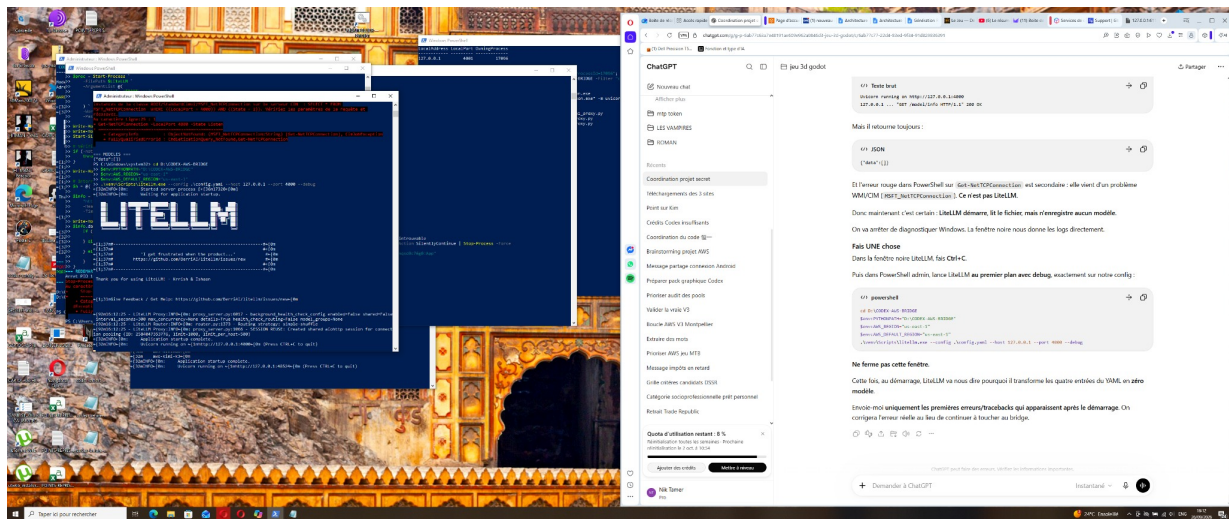
LiteLLM tourne sur 127.0.0.1:4000, mais les endpoints de diagnostic restent vides.

```
GET /v1/models -> {"data":[],"object":"list"}
GET /model/info -> {"data":[]}
```

Python parse pourtant le YAML et voit les quatre routes. Continuer a modifier GPT-OSS ou Codex aurait donc masque la panne reelle : la couche LiteLLM du bridge.

5. Le raisonnement bascule : isoler la chaine critique

Le debug confirme que LiteLLM démarre sans erreur fatale évidente tout en n'exposant aucun modèle. La bonne décision n'est plus de réparer toute l'infrastructure historique avant de reprendre la production, mais de découpler GPT-OSS de la couche malade.



Session de debug LiteLLM : le serveur démarre, mais la model list n'est toujours pas exploitable.

Decision : ne pas casser Kim, ne pas dependre de 4000/4001 pour GPT-OSS, creer un pont dedie sur 4010.

Agents historiques : Codex -> 4001 -> 4000 -> AWS
GPT-OSS 120B : Codex -> 4010 -> Mantle -> GPT-OSS

6. Le pont 4010 : coeur de Frankenstein

Le nouveau pont est volontairement minimal. FastAPI recoit les requetes, httpx les transmet a AWS et le proxy impose le vrai identifiant Mantle. Cette simplicité rend la chaîne observable et testable.

```
MANTLE = "https://bedrock-mantle.us-east-1.api.aws/v1"
MODEL = "openai.gpt-oss-120b"

if path == "responses":
    data["model"] = MODEL
```

Provider Codex

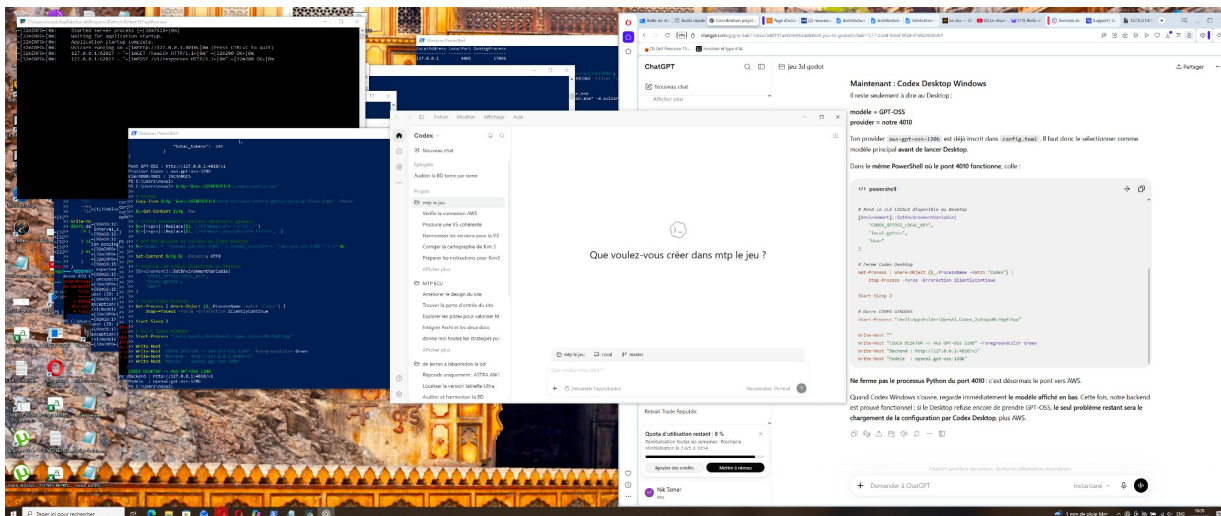
```
[model_providers.aws-gpt-oss-120b]
name = "AWS GPT-OSS 120B"
base_url = "http://127.0.0.1:4010/v1"
env_key = "CODEX_GPTOSS_LOCAL_KEY"
wire_api = "responses"
requires_openai_auth = false
```

Le proxy lit la vraie cle AWS depuis AWS_BEDROCK_API_KEY. La cle locale du provider Codex est separee. Aucune cle AWS n'a besoin d'etre codee en dur dans le fichier Python.

Le pont expose egalement /health et /v1/models afin de verifier localement le backend avant de declencher une inference facturable.

7. Preuve : Frankenstein vit

Le test final a d'abord confirme le pont local : status ok, backend AWS Bedrock Mantle, modele openai.gpt-oss-120b. Une vraie requete /v1/responses a ensuite ete envoyee. AWS a retourne une reponse avec un identifiant resp_..., status completed et le modele exact GPT-OSS 120B.



Validation finale : le serveur 4010 journalise les requetes, Codex Desktop est ouvert et PowerShell affiche la reponse AWS.

Preuve quantitative : 80 tokens d'entree, 64 tokens de sortie, 62 reasoning tokens, 144 tokens au total. L'output texte du micro-test etait vide parce que le budget de sortie etait presque entierement consomme par le raisonnement. Le statut completed et les compteurs d'usage prouvent toutefois l'execution effective du modele.

8. Carte d'identite technique et exploitation

Composant	Valeur
Interface	Codex Desktop Windows
Modele	openai.gpt-oss-120b
Cloud	AWS Bedrock Mantle
Region	us-east-1
Endpoint local	http://127.0.0.1:4010/v1
API	Responses
Serveur local	FastAPI + Uvicorn
Client	httpx
Authentification AWS	AWS_BEDROCK_API_KEY
Provider Codex	aws-gpt-oss-120b
Bridge historique	4000/4001 conserve separement

Routine de controle

```
GET http://127.0.0.1:4010/health
-> status: ok
-> backend: AWS Bedrock Mantle
-> model: openai.gpt-oss-120b
```

Chaque appel /v1/responses peut etre facture par AWS. Le processus 4010 doit rester actif pendant une session GPT-OSS. Toute cle AWS affichee accidentellement dans une capture ou un journal doit etre consideree comme exposee et remplacee.

Conclusion

Frankenstein marche parce que chaque couche a finalement ete isolee et prouvee separement. Codex fournit l'atelier. Le proxy 4010 fournit le pont. Mantle fournit la route cloud. GPT-OSS 120B fournit le cerveau. Le bridge historique reste disponible pour les autres agents.